Security Self-Assessment Report: Permission Tiers & Known-Risk Analysis

Competition: 2026 Shenzhen International FinTech Competition (FinTechathon) - International Track
Topic: Topic F - Agent Wallet (Autonomous On-chain Wallet)
GitHub Repository: https://github.com/Crux-15/agent-wallet

1. Permission-Tier Implementation

The PolicyEngine (policy.py) enforces three distinct operational permission tiers, selectable via the POLICY_TIER environment variable or live in the Streamlit Web Control Center sidebar. All hard limits (allowlist, 0.01 ETH per-tx cap, 0.05 ETH 24h cap, 0.002 ETH reserve floor, 50 Gwei gas ceiling) apply in all tiers:

- Tier 0 (READ_ONLY = 0): The agent can execute read-only portfolio, balance, limit, and dry-run simulation tools. All outgoing transfers (send_eth, batch_send_eth) are immediately denied.
- Tier 1 (APPROVE_ALL = 1): Every valid outgoing payment pauses for explicit human supervisor approval (status = 'needs_approval') regardless of how small the amount is.
- Tier 2 (LIMITED_AUTO = 2, Default): Micro-payments <= 0.002 ETH to allowlisted addresses execute autonomously; medium payments (> 0.002 ETH up to 0.01 ETH) require human supervisor approval.

```
Empirical Verification Output (evidence/tier_demo_output.txt):
Tier 0 (read-only)             result=denied   human_asked=no  risk=1/100
Tier 1 (approve everything)    result=sent     human_asked=yes risk=1/100
Tier 2 (small = automatic)     result=sent     human_asked=no  risk=1/100
```

2. Implemented Defense-in-Depth Controls

1. Complete Key Isolation: The LLM agent has no tool or code path to read PRIVATE_KEY or sign transactions.
2. Self-Approval Immunity: Passing 'approved': True inside tool arguments is ignored by ToolExecutor (verified by unit test test_the_model_cannot_approve_itself).
3. Persistent 24-Hour Spend Tracking: Spend history is stored in logs/policy_state.json so restarting the application cannot reset daily spending limits.
4. Sanitized Telemetry: Audit logs (logs/audit.jsonl) record only error class names to prevent secret leakage.

3. Known-Risk List & Production Remediation Roadmap

1. Local Console / Web UI Approval Channel: Human approval currently relies on local terminal input or a local Streamlit button. Remediation: Use cryptographic out-of-band WebAuthn passkey or hardware wallet signatures.

2. Plaintext .env Key Storage: The testnet private key sits in a local git-ignored .env file. Remediation: Migrate key custody to an HSM, cloud KMS, or ERC-4337 Smart Account with scoped session keys.

3. Local JSON Spend State File: A local user with disk write access could modify logs/policy_state.json. Remediation: Enforce rolling spend limits directly on-chain via smart contract policy modules.

4. Static Environment Allowlist: Approved recipient addresses are loaded from .env. Remediation: Implement an admin-signed on-chain address book registry with time-locked additions.

5. Testnet-Only Validation: Tested on Ethereum Sepolia testnet. Remediation: Add Mainnet chain-reorg confirmation depth checks and Flashbots Protect RPC for MEV defense.

6. External Cloud LLM API: Free-tier Gemini API calls transmit prompts to Google servers. Remediation: Use enterprise zero-data-retention endpoints or locally hosted open-weights models.